Onyx — Developer Guide

Onyx · multi-process OS on Raspberry Pi 4

This guide explains how to **build** Onyx, how to **write an application** or a `/bin` **tool**, how to **extend the** `kapi` **ABI**, and the **conventions** + **pitfalls** to be aware of. For the details of how things work internally, see Kernel internals.

Contents

---

1. Prerequisites and toolchain

- **AArch64 bare-metal toolchain**: `aarch64-none-elf-` (GCC), used under **WSL** on a Windows development machine.
- **Circle**, cloned **next to** the kernel tree (in `Zircon/circle/`).
- `make`, `cp`, `mkdir` (standard Unix tools, via WSL).
- A **FAT32 SD card** and a **Raspberry Pi 4** to run on (there is no QEMU `raspi4b` in the reference dev environment; bring-up is done directly on hardware).

> **Important — which Circle?** The Circle to modify is **`Zircon/circle`** (the clone next to the kernel), **not** any `C:\Temp\circle`. If you patch Circle (e.g. a keymap), do it in `Zircon/circle` and rebuild the affected library.

2. Building Circle (once)

```
# From the repo root (Zircon/), clone Circle next to the kernel:
git clone --recurse-submodules https://github.com/rsta2/circle.git

# Circle was checked out as CRLF on Windows -> renormalize to LF once:
git -C circle config core.autocrlf false && git -C circle rm --cached -rq . && git -C circle reset --hard

# Configure for RPi 4 / AArch64. DEPTH=32 is REQUIRED: our GImage renders 32-bit
# pixels (Circle defaults to 16).
```

```
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f

# Build the libraries used by the kernel:
(cd lib && make -j4) \
  && (cd lib/sched && make -j4) \
  && (cd lib/fs && make -j4 && cd fat && make -j4) \
  && (cd lib/usb && make -j4) \
  && (cd lib/input && make -j4) \
  && (cd addon/SDCard && make -j4) \
  && (cd addon/fatfs && make -j4)
cd ..
```

> If you change `DEPTH` later, run `make clean` in `circle/lib` before rebuilding (the `.o` files do not depend on `Config.mk`).

The libraries linked by the kernel (cf. `kernel/Makefile`): `libsched.a`, `libfatfs.a`, `libsdcard.a`, `libfs.a`, `libusb.a`, `libinput.a`, `libcircle.a`.

3. Building the kernel and applications

From `kernel/`:

```
cd kernel
make            # -> kernel8-rpi4.img, THEN builds the apps (../user)
```

The default target (`all`):

1. compiles our sources + links against the Circle libs → **`kernel8-rpi4.img`**;
2. triggers `make -C ../user`, which builds **all the apps** (`user/*.elf`) and the **`/bin` tools** (`user/bin/*.elf`).

> **Kernel → apps order.** Since the apps go through the **fixed-address ABI table** and are **not** linked against the kernel's addresses, they no longer depend on the kernel build: a *kernel-only* change (that respects the ABI) does **not** require recompiling the apps. `make` rebuilds them anyway for convenience.

Details of the kernel wiring (which Circle files we replace, the order of the `-I compat -I include` include path before `circle/include`): see Kernel internals §1 and `kernel/Makefile`.

4. Deploying to the SD card

```
cd kernel
make stage      # copie l'image + chaque app + chaque outil vers ../sdcard/
```

`make stage`:

- copies `kernel8-rpi4.img` → `sdcard/`;
- for each `../user/<name>.elf`: creates `sdcard/apps/<name>.app/` and copies the ELF into it as **`main.elf`** (the "Onyx" layout);
- copies each `../user/bin/<tool>.elf` → `sdcard/bin/<tool>.elf`.

Then copy **all** of the contents of `sdcard/` onto a **FAT32** card and boot the Pi 4. See the user guide for details about the card and the configuration files.

5. The application model

A Onyx application is a **single `.c` file** compiled into a **freestanding ELF** and run in **EL1** in its own page table.

Runtime (`user/crt0.S`):

```
_start:
    stp x29, x30, [sp, #-16]!   /* the stack is already set up by the kernel */
    bl  main                    /* call main() */
    ldp x29, x30, [sp], #16
    ret                         /* return -> the kernel terminates the task */
```

No argv is passed via the stack: `main()` takes no arguments. An app retrieves its argument line via `kapi_get_args(buf, size)`, and exits early with `kapi_exit(status)`.

Linking (`user/user.ld`): everything is linked at **`USER_VA_BASE` = 8 GB** (`. = 0x200000000;`). This is **not PIE**. The RX (`.text`/`.rodata`) and RW (`.data`/`.bss`) sections are aligned on **64 KB** (`-z max-page-size=0x10000`) so that the loader maps them onto distinct pages. **No kernel symbol** is resolved at link time: everything goes through the ABI table.

Compilation flags (`user/Makefile`):

```
-ffreestanding -nostdlib -fno-pic -fno-pie -mgeneral-regs-only \
-O2 -Wall -Wextra -fno-stack-protector -I../kernel/include
```

- `-ffreestanding -nostdlib`: **no libc**. No `printf`, `malloc`, `string.h`... use `applib.h` (§8) and static/local buffers.
- `-mgeneral-regs-only`: **no FP/SIMD** — the kernel trap frame does not save the floating-point registers. Do **integer / fixed-point arithmetic** (see `tinycalc.c`, `mandelbrot.c`).
- `-I../kernel/include`: to include `<kern/kapi_abi.h>` (the shared ABI structure).

To add an app, create `user/<name>.c` and **add `<name>.elf` to the `PROGS` variable** of `user/Makefile` (and `<tool>.elf` to the `PROGS` of `user/bin/Makefile` for a tool).

6. Writing a graphical application

Minimal skeleton (window with kernel-managed widgets):

```
#include "kapi.h"

static void on_ok (unsigned long sender, int ev, long value)
{
    (void) sender; (void) ev; (void) value;
    kapi_widget_set_text (g_label, "cliqué !");
}

static unsigned long g_label;

int main (void)
{
    /* Le canvas est mappé à 12 Go ; fb[y*w + x] = 0x00RRGGBB. */
    unsigned *fb = kapi_create_window (300, 200, "exemple");

    g_label = kapi_add_label  (10, 10, 200, 16, "prêt");
    (void)    kapi_add_button (10, 40, 80, 28, "OK", on_ok);
```

```
    kapi_wait_for_exit ();   /* pompe les événements à ~60 fps jusqu'à fermeture */
    return 0;
}
```

Key points:

- `kapi_create_window(w, h, title)` returns a pointer to the **canvas** (pixel buffer of `0x00RRGGBB`, width `w`). The app draws directly into it (no per-pixel call). The variant `kapi_create_window_ex(x, y, w, h, title, flags)` is for explicit placement and `WIN_FLAG_BORDERLESS`.
- **Kernel widgets**: `kapi_add_button/label/checkbox/textbox/progress/slider/textarea/scrollbar_v/scrollbar_h/icon(...)` return an `unsigned long` handle. Manipulate them with `kapi_widget_set_text/get_text`, `get_checked`, `get/set_value`, `set_rect` (move/hide by setting `w=h=0`), `set_icon`.
- **Event loop**: either `kapi_wait_for_exit()` (blocking, simple), or your own loop `while (!kapi_should_exit()) { ...; kapi_pump_events(); kapi_present(); kapi_msleep(16); }` when you animate the canvas yourself.
- **App-drawn UI**: to draw text over your canvas, use `kapi_draw_text(x, y, s, color)` + `kapi_font_width/height()`. Capture the keyboard with `kapi_set_key_handler(fn)` (`GUI_EVENT_KEY` events, `KEY_*`/ASCII values) and the "outside-widget" clicks with `kapi_set_click_handler(fn)` (`GUI_EVENT_CANVAS_CLICK` / `..._MOTION`, coordinates encoded in `value`). Cursor position relative to the window: `kapi_cursor_pos(&x, &y)`.
- **Cooperative**: your app **must yield** regularly (`present`/`msleep`/ `pump_events`/`wait_for_exit`), otherwise it freezes the whole system.

See the demos `demoD.c` (widget gallery), `demoE.c` (textarea + scrollview), and the apps `tinypad.c`, `paint.c`, `mandelbrot.c` for complete examples.

7. Writing a `/bin` tool

A `/bin` tool follows the **same EL1 app model** but reads `stdin`, writes `stdout`, and exits (no window). It is composable via the terminal's pipes.

```
#include "kapi.h"
#include "applib.h"     /* ax_puts, ax_putln, ax_strlen, ax_itoa, ... */

int main (void)
{
    char args[128];
    kapi_get_args (args, sizeof args);   /* la ligne d'arguments (chaîne) */

    /* Lire stdin et le réécrire en majuscules, par exemple : */
    char buf[256];
    int n;
    while ((n = kapi_stdin_read (buf, sizeof buf)) > 0)
    {
        for (int i = 0; i < n; i++)
            if (buf[i] >= 'a' && buf[i] <= 'z') buf[i] -= 32;
        kapi_stdout_write (buf, (unsigned) n);
    }
    return 0;   /* code de sortie récupérable via kapi_wait dans l'appelant */
}
```

- `kapi_get_args(buf, size)`: the entire argument line as **a single string** separated by spaces (there is no `argv` array; parse the first word yourself, etc.).
- `kapi_stdin_read(buf, n)`: reads the task's stdin (`0` = EOF). `kapi_stdout_write`: writes stdout.
- To read a file passed as an argument: `kapi_open/read/close`. To list a directory: `kapi_opendir/readdir/closedir`.

The existing tools to study: `ls`, `cat`, `grep`, `wc`, `echo`, `page`, `rm`, `mkdir`, `touch`, `cp`, `mv`, `ps`, `kill`, `run`, `keyb` (in `user/bin/`).

8. The `applib.h` library

`user/applib.h` is **header-only** (no libc). It provides:

- **Strings**: `ax_strlen`, `ax_streq`, `ax_strcat(dst, cap, &pos, src)` (concat without overflow), `ax_app_path(dst, cap, name, suffix)` (builds `SD:apps/<name><suffix>`), `ax_itoa(v, buf)`, `ax_fmt2(d, v)` (2-digit decimal).
- **Console**: `ax_puts(s)`, `ax_putln(s)` (to stdout).
- A minimal **`.ini` reader**: `app_ini_load("config.ini")` (from the app's folder via `kapi_app_dir`) or `app_ini_load_path("SD:skins/theme.txt")`; then `app_ini_get(section, key, default)` and `app_ini_get_int(...)`. Sections `[xxx]`, lines `key=value`, comments `;`/`#`.
- **App-drawn widgets** (not kernel widgets): `ax_dropdown` (drop-down list) and `ax_colorpick` (palette-based color picker), with `*_draw(...)` and `*_click(...)`. The app draws them in its canvas and routes the clicks via its `set_click_handler`. Used by `theme.c` and `mandelbrot.c`.

9. Packaging an app: `.app`, icons, `config.ini`

Layout of an application on the card (produced by `make stage`):

```
SD:apps/<nom>.app/
  main.elf       l'ELF de l'app (obligatoire)
  icon.bmp       icône 40×40, BMP 24 bpp ; le magenta 0xFF00FF est transparent (optionnel)
  config.ini     configuration de l'app, lue via app_ini_load() (optionnel)
```

The **app name** is the base name of the `.app` folder (without the suffix). That is what you put in `autostart.txt`/`quicklaunch.txt` and what `kapi_list_apps` returns.

Icons — `tools/gen_assets.py` procedurally generates the 40×40 BMPs (BGR bottom-up, 4-byte padding) for all the apps (a document for `tinypad`, a calculator for `tinycalc`, a glider for `life`, etc.) and the "9 squares" glyph of the *apps* button in the panel. `tools/preview_icons.py` produces a PNG preview montage. Workflow: run `gen_assets.py` (writes the `icon.bmp` files into `sdcard/apps/<name>.app/`), then `preview_icons.py` to check visually.

`config.ini` — example read by `inidemo` / `voronoy` / `panel`:

```
; commentaire
greeting = bonjour
[app]
name = Mon App
version = 1
[display]
barwidth = 40
```

Screenshots and documentation

- **Screenshots (simulated but faithful)**: `tools/screenshot/render.py` loads the **real** skins (`SD:skins/wings/button/closebgs.bmp`), the **real** font `circle/lib/font8x16.cpp` and the icons `SD:apps/<name>.app/icon.bmp`, and reproduces the drawing routine of each app — so the output matches the real OS. Run `python tools/screenshot/render.py` → writes the PNGs into `screenshots/` (`desktop.png` = overview, plus one screenshot per app, window only on a transparent background). **To add an app**: write `app_<name>()` that reproduces the `redraw()` of `user/<name>.c` (same colors/coords), add `("<name>", app_<name>)` to the `APPS` list, and re-run.
- **Word/PDF exports**: `docs/build_docs.py` converts each `.md` in `docs/` into `.docx` (via `pandoc`) then into `.pdf` (via Word/`docx2pdf`), in `docs/exports/`. Run `python docs/build_docs.py` after any modification to the `.md` files. Prerequisites (once): `pip install python-docx docx2pdf pypandoc_binary`.
- **Reminder of the project rule** (see `CLAUDE.md`): any modification of a `kapi` function or of an app **updates the docs in the same go**, regenerates the affected screenshot if the visual changes, then regenerates the exports.

10. Extending the `kapi` ABI

The ABI is **append-only**. To add a kernel function callable by apps, there are **5 points** to touch (all in the same direction, at the end):

1. **`kernel/include/kern/kapi_abi.h`** — add the function pointer **at the end** of `struct TKApiTable` (never in the middle, never reorder), with a version comment, and **increment** **`KAPI_ABI_VERSION`**.

   ```
   // --- v22 additions ---   (next free version; v21 added the TCP sockets)
   int (*ma_fonction) (int arg);
   ```

2. **`kernel/sys/kapi.cpp`** — implement `extern "C" int kapi_ma_fonction(int arg)`. It runs in the app's context (use `CurrentAS()` for the current address space if needed).

3. **`kernel/sys/kapitable.cpp`** — declare the `extern "C"` prototype and **assign the pointer** in `KApiTableInit()`: `t->ma_fonction = kapi_ma_fonction;`.

4. **`user/kapi.h`** — add the inline wrapper:

   ```
   static inline int kapi_ma_fonction (int arg) { return KT->ma_fonction (arg); }
   ```

5. Rebuild (`make`). The apps that want the new function use it; the old ones keep working (they ignore the new field).

> **Golden rule:** never change the signature or the order of an existing field. If some semantics must change, add a **new** entry. An app can query `((const struct TKApiTable *)KAPI_TABLE_VA)->version` to find out what is available.

If you add a new **GUI event** or a **window flag**, keep the values synchronized between `kernel/gui/window.h` and the `#define`s in `user/kapi.h` (commented "must match").

11. Coding conventions

- **Kernel (C++)**: Circle style. `CXxx` classes, `m_Xxx` members, CamelCase methods, `boolean`/`TRUE`/`FALSE` and Circle's `u8/u16/u32/u64` types. No exceptions or RTTI. `new`/`delete` go through Circle's heap.

- **Userland (C)**: freestanding C. `ax_` prefix for the `applib.h` helpers. Globals `g_xxx`. Bounded static buffers (no dynamic allocation on the app side in general).
- **kapi**: `extern "C"` functions named `kapi_xxx` on the kernel side; inline wrappers `kapi_xxx` on the app side.
- Respect the **comment density** and the **idiom** of the file you are modifying.
- **Git**: commit into the **Onyx repo** explicitly — the current working directory (cwd) drifts; a bare `git` may land in the wrong repo. (Note: `circle/` is not committed in this repo.)

12. Debugging on hardware

Bring-up is done **directly on the Pi 4** (no QEMU raspi4b). Tools:

- **On-screen exception dump**: an EL1 synchronous fault (or an EL0 fault) paints a panic + register dump on the HDMI framebuffer (`PanicToScreen` + Circle's handler). Note the `ELR` (faulting PC).
- **`addr2line`**: `aarch64-none-elf-addr2line -e kernel8-rpi4.elf <ELR>` to locate the faulting line (keep the unstripped `.elf` next to the `.img`).
- **Serial console**: `config.txt` must have `enable_uart=1` (PL011 clock). The boot log goes **also** to the HDMI screen (`CScreenDevice`) so it is readable without a serial cable.
- **Post-mortem console**: on an app's exit, the compositor clears and the logger is shown on the framebuffer (see Kernel internals §11).

13. Known pitfalls

- **Do not save FP/SIMD.** `-mgeneral-regs-only` is mandatory on the app side; use fixed-point. An accidental floating-point use would corrupt the state on a context switch.
- **L3 tables shared with the kernel.** On the kernel side, never free an L3 table from the user area without checking that it is not shared with the kernel's L2 (cf. Kernel internals §4). Otherwise: global corruption.
- **Do not free the ABI table page.** It is global to the kernel; the destruction of an address space already skips it.
- **`DEPTH=32` for Circle.** `GImage` renders 32-bit; forgetting `-d DEPTH=32` (or changing `DEPTH` without `make clean` in `circle/lib`) gives wrong colors/breakage.
- **Cooperative.** Any app loop without `present`/`msleep`/`yield` freezes the system (no preemption).
- **Circle LF renormalization.** On Windows, Circle is checked out in CRLF; renormalize once (cf. §2) otherwise the build breaks.
- **The right Circle.** Patch `Zircon/circle`, not another clone.